Hustronix OS — Complete Technical Guide

Version: 1.0 · **Generated:** June 6, 2026
Audience: Founders and operators who want every layer explained in plain language, with full technical detail for engineers.

How to read this document

If you want to understand…	Start here
What the product *is* in one page	§1 Executive Summary
How data flows from Slack/GitHub into decisions	§5 Data Flow
Every screen in the app	§7 Frontend Routes
Every API endpoint	§8 Backend API
Where data is stored	§9 Database
Login, permissions, security	§10 Auth & Security
AI chat and the coordinator	§11 Chat & AI
Rules that enforce capacity and shadow work	§12 Auditor
Search, signals, embeddings	§13 Intelligence
Slack, GitHub, Google, etc.	§14 Integrations
Where it runs in production	§16 Deployment
All configuration knobs	§17 Environment Variables
What’s finished vs partial	§18 Product Maturity

Core idea in one sentence: Hustronix is an **operating system for decisions and execution** — external signals (Slack, GitHub, email, meetings) land in a **database ledger**, **deterministic rules** enforce capacity and policy, and an **AI chat interface** helps you draft and navigate that ledger without replacing it.

1. Executive Summary

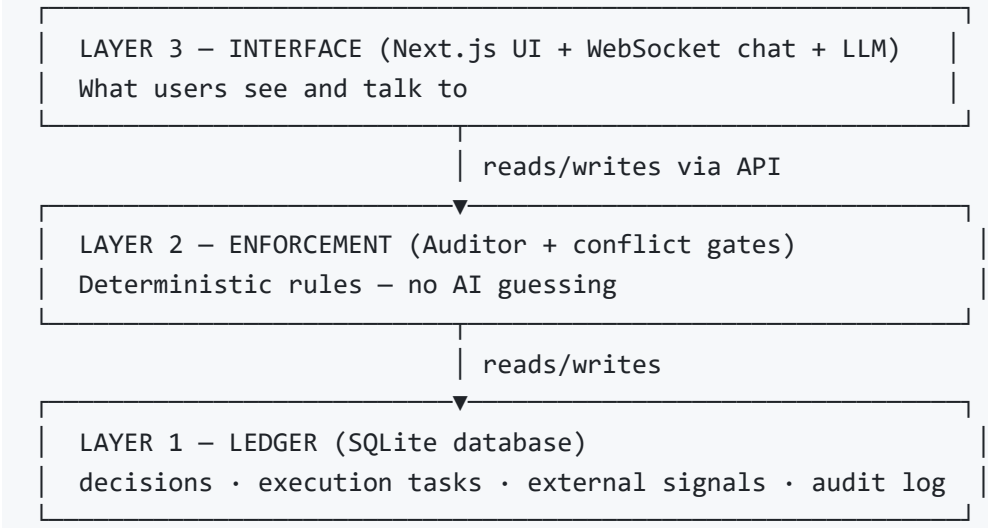
What Hustronix is

Hustronix is a **decision and execution platform** for teams that need to:

1. **Commit** to a small number of strategic decisions (the "vow").
2. **Track execution** as tasks under those decisions.
3. **Ingest signals** from tools like Slack, GitHub, Google Calendar, and meetings.
4. **Enforce rules** automatically — shadow work, capacity limits, conflicting decisions.
5. **Talk to an AI coordinator** that reads the ledger and helps you act — but does not silently override rules.

Think of it as **Mission Control for organizational focus**: not another chatbot, but a system where the **database + auditor** are the source of truth and the LLM is an interface.

The three layers (most important mental model)



File anchors: Ledger = `apps/api/src/database.ts` · Auditor = `apps/api/src/auditor.ts` · Coordinator = `apps/api/src/agent.ts` · UI = `apps/web/`

Technology stack (simple glossary)

Term	What it means for Hustronix
Monorepo	One Git repository containing API, web app, shared code, and deployment configs
Node.js	JavaScript runtime that powers the backend server
Express	Web framework that handles HTTP API requests
Next.js	React framework for the web UI (pages, routing, styling)
SQLite	Single-file database stored at `./data/hustronix.db` (local) or mounted volume (production)
WebSocket	Persistent connection used for real-time chat streaming
LLM	Large language model (Claude by default) powering the chat coordinator
OAuth	Standard way users connect Slack, GitHub, Google, etc.
Unified server	Production mode: one process serves API + chat + website on one port

---

2. Repository Structure

```
Hustronix OS/
├── apps/
│   ├── api/                Backend: Express, WebSocket, agents, SQLite
│   │   └── src/
│   │       ├── server.ts           HTTP routes (main API surface)
│   │       ├── unified-server.ts   Production: API + Next.js together
│   │       ├── database.ts         Schema, queries, migrations
│   │       ├── agent.ts            Main AI coordinator
│   │       ├── auditor.ts          Rule engine (shadow work, capacity)
│   │       ├── bootstrap.ts        Startup: seeds, cron, WS wiring
│   │       ├── auth/               Login, OAuth, sessions, 2FA
│   │       ├── integrations/       Slack, GitHub, Google, etc.
│   │       ├── intelligence/       Search, embeddings, auto-linking
│   │       ├── notifications/      In-app, email, Slack delivery
│   │       └── security/           RBAC, CSRF, rate limits, audit
│   └── web/                Frontend: Next.js App Router
```

```
│       └── src/app/          Pages: mission-control, chat, signals, etc.
├── packages/
│   └── shared/               Shared TypeScript types between API and web
├── infra/
│   ├── docker/               Dockerfile + docker-compose
│   ├── fly/                  Fly.io deployment config
│   ├── azure/                Azure Container Apps (production target)
│   └── db/postgres/          Future PostgreSQL schema (not runtime yet)
├── netlify-landing/          Static marketing site (early access only)
├── docs/                     Architecture and security documentation
├── ops/scripts/              QA, security checks, PDF export
├── data/                     Local SQLite file (gitignored except .gitkeep)
├── .env                      API secrets (copy from .env.example)
└── package.json              Root scripts: dev, build, deploy
```

---

3. How the App Runs

Development (two processes)

When you run `npm run dev`:

Service	Port	What it does
API + WebSocket	3005	Handles `/api/*` and chat WebSocket
Next.js web	3000	Serves the UI; proxies `/api/*` to port 3005

The browser usually opens **http://localhost:3000**. API calls go to the same origin and get proxied to the backend.

Key files: `apps/web/next.config.ts` (proxy rules) · `apps/web/src/lib/api.ts` (how the UI calls the API)

Production (unified — one process)

When you run `npm run build` then `npm run start:unified`:

- **One Node process** on port 3000 (or `PORT` env var)
- Serves REST API, WebSocket, and Next.js pages together

- **File:** `apps/api/src/unified-server.ts`

This is how Docker, Fly.io, and Azure deployments run.

Boot sequence (what happens when the server starts)

1. Load environment from `.env`
2. Open SQLite database, run schema migrations if needed
3. Seed `system` user if missing
4. Attach WebSocket handler to HTTP server
5. Start **auditor cron** (every 5 minutes — safety net)
6. Start **integration workers** (OAuth refresh, Google/Slack polling, notification digest)
7. Optionally seed demo data (decisions + sample signals)

File: `apps/api/src/bootstrap.ts`

---

4. Request Flow (Browser → Database)

```
User clicks "Commit Decision" in Mission Control
        │
        ▼
Next.js page (MissionControlClient.tsx)
        │
        ▼  POST /api/decisions/commit  (cookie: hustronix_session)
Express server.ts
        │
        ▼  authMiddleware → resolves userId from session
        │
        ▼  requirePermission('decision:commit')
        │
        ▼  database.transitionDecisionState(ACTIVE)
        │     └── findOtherActiveDecision() — blocks if another ACTIVE exists
        │
        ▼  auditor.runAfterLedgerMutation()
        │
        ▼  JSON response + optional WebSocket broadcast
```

Every mutating API route follows this pattern: **authenticate → authorize → database → auditor → respond**.

---

5. Data Flow — Signals to Execution

This is the product's central loop:

```
External world                    Hustronix ledger                UI
──────────────                    ────────────────                ──

Slack message  ──webhook──►  external_signals (UNVERIFIED)
GitHub PR      ──webhook──►       │
Meeting notes  ──API──────►       ▼
                             Intelligence pipeline:
                             · detect intent/risk (LLM)
                             · embed for search
                             · auto-link (if confident)
                                  │
                                  ▼
                             Signals UI (/signals)
                             User links signal → task
                                  │
                                  ▼
                             external_signals (LINKED)
                             linked_execution_id → execution_ledger
                                  │
                                  ▼
                             Auditor checks shadow work,
                             capacity, grace periods
```

Key principle: Ingestion **never** silently creates execution work. Linking is **explicit** (user or high-confidence auto-linker).

Docs: `docs/architecture-enforcement.md` · **Code:** `apps/api/src/intelligence/index.ts`

---

6. Multi-Tenancy

Each customer organization is scoped by **`user_id`** on rows:

- `external_signals.user_id` — signals belong to one tenant
- `decision_ledger` / `execution_ledger` — scoped to the org owner
- Settings like `org.owner_user_id` and `org.role` define RBAC within a tenant

The `system` automation principal exists for scripts but does **not** receive cross-tenant signal reads.

---

7. Frontend Routes

All pages live under `apps/web/src/app/`.

Public / marketing

URL	Component	Purpose
`/`	`LandingPage`	Product marketing homepage
`/login`	`LoginClient`	Sign in (password, OTP, OAuth)
`/signup`, `/register`	`SignupClient`	Create account
`/contact`	`ContactForm`	Contact form
`/privacy`, `/terms`, `/cookies`	Legal pages	Legal content

Authenticated dashboard

Wrapped in `(dashboard)/layout.tsx` — sidebar, auth gate, onboarding check.

URL	Component	API / behavior
`/mission-control`	`MissionControlClient`	`GET /api/mission-control/summary`, decisions list, WebSocket `COMMIT_DECISION`
`/execute`	`ExecuteClient`	Execution ledger, capacity, state transitions
`/signals`	`SignalsClient`	Signal triage by state, link to tasks

URL	Component	API / behavior
`/chat`	`ChatClient`	WebSocket chat with AI coordinator
`/reports`	`ReportsClient`	`GET /api/metrics/reports` — charts
`/teams`	`TeamsClient`	Team roster, bar/donut charts
`/settings`	`SettingsClient`	Org settings, notifications, integration toggles
`/profile`	`ProfileClient`	Profile, API keys, active sessions
`/guide`	`GuideClient`	In-app product walkthrough
`/onboarding`	`OnboardingClient`	First-run org setup

Next.js API routes (server-side)

Route	Purpose
`/api/contact`	Proxies marketing contact to backend
`/api/early-access/leads/submit`	Waitlist submission
`/api/early-access/leads/list`	Admin list (protected)
`/oauth/[provider]`	OAuth redirect helper

Separate marketing site

`netlify-landing/` — static site for demand validation only (early access + contact). No product app. Hosted on Netlify at `hustronix.online`. Regenerate with `npm run export:netlify-landing`.

---

8. Backend API — Complete Route Reference

Primary file: **`apps/api/src/server.ts`** (+ integration modules mounted at bottom).

Legend: 🔓 public · 🔐 requires login · 🛡️ requires specific permission

Health & marketing 🔓

Method	Path	Purpose
GET	`/api/health`	Liveness: DB ok, uptime, LLM config (no secrets)
POST	`/api/early-access`	Waitlist signup → `early_access_leads`
POST	`/api/contact`	Contact form → `marketing_contact_messages`

Webhooks (signature-verified, no session) 🔓

Method	Path	Source
POST	`/api/integrations/slack/events`	Slack Events API
POST	`/api/ingest/github`	GitHub webhooks
POST	`/api/ingest/github/:tenant`	GitHub (tenant-scoped)
POST	`/api/ingest/linear/:tenant`	Linear
POST	`/api/ingest/jira`, `/api/ingest/jira/:tenant`	Jira
POST	`/api/ingest/crm/:vendor`	Generic CRM
POST	Google/Microsoft ingest routes	Calendar, Teams, Outlook

Authentication 🔓 / 🔐

Method	Path	Purpo
GET	`/api/auth/providers`	Enabled lo methods
POST	`/api/auth/otp/request`	Send emai
POST	`/api/auth/register`	Password registratio

Method	Path	Purpc
POST	`/api/auth/login`	Password → session
POST	`/api/auth/login/totp`	TOTP seco factor
GET	`/api/auth/totp/status`	2FA enable
POST	`/api/auth/totp/setup|confirm|cancel|disable`	Manage 2 `security:`
POST	`/api/auth/logout`	Revoke se
GET/POST	`/api/auth/google/*` , `/api/auth/github/*`	Social **logi** OAuth
POST	`/api/auth/password`	Change password
GET/POST/DELETE	`/api/auth/api-keys`	Automatic tokens `hx_live_...`
GET/DELETE	`/api/auth/sessions`	List/revoke sessions
POST	`/api/auth/sessions/revoke-all`	Log out everywher

Integration OAuth (connect tools) 🔐

Method	Path	Purpose
GET/DELETE	`/api/oauth/:provider/*`	Slack, GitHub, Google, Microsoft, Notion, Jira, Linear
GET	`/api/oauth/connections`	List connected integrations

File: `apps/api/src/auth/oauthRoutes.ts`

Decisions & execution 🔐

Method	Path	Permission	Purpose
GET	`/api/mission-control/summary`	read	Dashboard aggregate
GET	`/api/decisions`	read	List decisions
POST	`/api/decisions`	write	Create DRAFT decision
POST	`/api/decisions/commit`	`decision:commit`	Activate decision (conflict gate)
GET	`/api/execution/ledger`	read	Task graph
GET	`/api/execution/capacity`	read	Capacity utilization
POST	`/api/execution/transition`	`execution:write`	Move task state
GET	`/api/execution/signals`	read	Signals list
GET	`/api/execution/signals/by-state`	read	Signals grouped by state
POST	`/api/execution/link`	`signal:write`	Link signal → task
POST	`/api/execution/signals/kill`	`signal:write`	Dismiss signal
POST	`/api/audit/run`	`audit:run`	Manual auditor sweep

Intelligence & operations 🔐

Method	Path	Purpose
GET	`/api/intelligence/context`	Decision context for UI/agent

Method	Path	Purpose
GET	`/api/intelligence/search`	Hybrid full-text + embedding search
POST	`/api/intelligence/relink`	Re-run auto-linker
POST	`/api/intelligence/warm-embeddings`	Precompute embeddings
POST	`/api/ingest/manual`	Manual signal insert
POST	`/api/ingest/meeting`	Meeting transcript → signals
GET	`/api/operational/drift`	Execution drift query
GET	`/api/operational/overloaded`	Overloaded owners
GET	`/api/operational/conflicts`	Conflicting work
GET	`/api/operational/related`	Related signals
GET	`/api/operational/search`	Operational search
GET	`/api/reports/weekly`	Weekly drift report
GET	`/api/reports/health`	Operational health report

User, settings, metrics 🔐

Method	Path	Purpose
GET/PATCH	`/api/me`	Profile + allowlisted settings keys
GET	`/api/activity`	Activity feed
GET/POST	`/api/settings`	Org settings
POST	`/api/settings/bulk`	Bulk update (notifications, general)
GET	`/api/settings/capacity-summary`	Capacity rollup

Method	Path	Purpose
GET/POST	`/api/billing`	Billing settings (stub)
GET	`/api/metrics/reports`	Report metrics for charts
GET	`/api/metrics/teams`	Team metrics
GET/POST/DELETE	`/api/metrics/teams/roster/*`	Roster CRUD, import
GET	`/api/chat/sidebar`	Chat sidebar context
GET	`/api/connections`	Connection capability cards
POST	`/api/connections/sync`	Sync connections
GET	`/api/integrations/health`	Integration health status

Notifications 🔐

Method	Path	Purpose
GET	`/api/notifications`	In-app inbox
GET	`/api/notifications/unread-count`	Badge count
POST	`/api/notifications/:id/read`	Mark one read
POST	`/api/notifications/read-all`	Mark all read
POST	`/api/notifications/test`	Test delivery

Observability 🔐

Method	Path	Purpose
GET	`/api/audit-log`	Sovereign audit log 🛡️ `audit:run`
GET	`/api/learning-log`	Post-decision learning
GET	`/api/productivity`	Productivity metrics
GET	`/api/system/value-telemetry`	Internal telemetry

9. Database — SQLite Schema

File: `apps/api/src/database.ts`
Default path: `./data/hustronix.db` (env: `SQLITE_PATH`)

Core ledger (source of truth)

Table	Purpose	Key columns
`decision_ledger`	Strategic commitments	`id`, `user_id`, `vow`, `status` (DRAFT→ACTIVE→...), `outcomes`, `constraints`
`execution_ledger`	Tasks under decisions	`task_id`, `decision_id`, `owner`, `status`, `priority`, `weight`, `dependencies`
`external_signals`	Events from Slack/GitHub/etc.	`id`, `user_id`, `source`, `state` (UNVERIFIED→GRACE→LINKED/KILLED), `linked_execution_id`
`audit_log`	Auditor actions	`action`, `entity_type`, `details`, `timestamp`
`ledger_events`	Hash-chained tamper-evident log	Chained hashes for integrity
`owner_reputation`	Behavioral reputation	Per-owner scores

Identity & authentication

Table	Purpose
`users`	Accounts: username, email, password_hash, TOTP columns
`user_sessions`	Session tokens (hashed), expiry, device info

Table	Purpose
`api_access_tokens`	Automation keys (`hx_live_…`)
`api_keys`	Per-user LLM provider keys (legacy)
`oauth_identities`	Social login linkage (Google/GitHub login)
`oauth_login_states`	Login OAuth CSRF states
`email_otp_challenges`	Email one-time password flows
`login_totp_challenges`	TOTP step-up during login
`oauth_connections`	Encrypted integration tokens (Slack bot, GitHub, etc.)
`oauth_pending_states`	In-flight OAuth flows
`oauth_subscriptions`	Webhook subscriptions (Microsoft, Google)
`slack_event_receipts`	Slack event idempotency
`external_event_dedup`	Cross-provider deduplication

Configuration & SaaS

Table	Purpose
`settings`	Key/value per user — org name, role, notification prefs
`team_roster`	Teams page roster entries
`notifications`	In-app notifications + delivery flags
`early_access_leads`	Marketing waitlist
`marketing_contact_messages`	Contact form submissions
`messages`	Chat history (user, session_id, role, content)
`learning_log`	Post-decision learning entries
`security_events`	Security audit trail

Intelligence

Table	Purpose
`signals_fts`, `executions_fts`, `decisions_fts`	Full-text search mirrors (FTS5)
`entity_embeddings`	Vector embeddings for hybrid search
`meeting_ingestions`	Meeting ingest metadata

Key relationships

```
decision_ledger (1) ──< execution_ledger (many)
                              ▲
                              │ linked_execution_id
external_signals ─────────────┘

Rule: at most ONE decision with status ACTIVE per user_id
      (enforced in transitionDecisionState)
```

Future: PostgreSQL

Schema exists at `infra/db/postgres/`. Migrate with `npm run db:pg:migrate`. **Runtime still uses SQLite** until cutover is complete.

---

10. Authentication & Security

How login works

1. User submits credentials at `/login`
2. API validates (password bcrypt, OTP, or OAuth)
3. API creates row in `user_sessions`, sets cookie `hustronix_session`
4. Browser sends cookie on every request (`credentials: "include"`)
5. `authMiddleware` resolves `userId` and attaches to request

Auth resolution order (`apps/api/src/middleware/auth.ts`):

1. `Authorization: Bearer hx_live_…` → API access token
2. Cookie `hustronix_session` → session table
3. Legacy cookie `hustronix_uid` (if `ALLOW_LEGACY_UID_COOKIE=1`)
4. Headers `x-api-key` + `x-user-id` (automation scripts)
5. Loopback automation (dev only, with `ALLOW_LOOPBACK_AUTH=1`)

Roles & permissions (RBAC)

File: `apps/api/src/security/authorization.ts`

Role	Typical access
owner	Full control — matches `settings.org.owner_user_id`
admin	Most mutating operations
strategist	Decisions, execution, signals
operator	Execution transitions, signal linking
viewer	Read-only (default if no role set)
agent	Automation tokens

Example permissions: `decision:commit` , `execution:write` , `signal:write` , `security:manage` , `agent:invoke` , `audit:run`

Security middleware stack

Module	File	Function
Request context	`security/requestContext.ts`	Request IDs, structured logs
HTTP security	`security/httpSecurity.ts`	HTTPS enforcement, security headers, CSP
CORS	`security/cors.ts`	Explicit allowed origins, credentials

Module	File	Function
CSRF	`security/csrf.ts`	Origin/referer check on cookie writes
Rate limits	`middleware/rateLimit.ts`	Auth, API, marketing, AI tiers
Settings allowlist	`security/settings-keys.ts`	Only approved keys writable via PATCH /api/me
Ledger integrity	`security/ledgerIntegrity.ts`	Hash-chained `ledger_events`
Security events	`security/audit.ts`	Redacted security logging

Webhook security

- Raw body preserved before JSON parsing
- HMAC verification (GitHub, Slack signatures)
- Idempotency tables prevent duplicate processing

Encryption keys (production required)

Variable	Purpose
`OAUTH_TOKEN_ENCRYPTION_KEY`	Encrypts Slack/GitHub/etc. tokens at rest
`TOTP_ENCRYPTION_KEY`	Encrypts 2FA secrets
`MISSION_CONTROL_API_KEY`	Automation / script access

Generate with: `openssl rand -hex 32`

Full threat model: `docs/security-architecture.md`
Launch checklist: `npm run security:launch`

---

11. Chat & AI System

Two chat modes

Session prefix	Agent	Purpose
Normal UUID	`agent.ts` (coordinator)	Full ledger access, decision tools
`asst:*`	`assistant-agent.ts`	In-app guide only, memory tools

WebSocket protocol

Attachment: Same HTTP server as Express (`attachWebSocketServer` in `server.ts`)

Client → Server messages:

Type	Action
`load_session`	Fetch chat history
`text`	Send message → run agent
`new_session`	Create new chat session
`COMMIT_DECISION`	Activate decision via WS
`SYNC_PROTOCOLS`	Refresh connections manager
`CAPABILITY_OVERRIDE`	Admin override (restricted)

Server → Client messages:

Type	Meaning
`typing`	Agent is thinking
`text_chunk`	Streaming response token
`text`	Complete message
`history`	Past messages
`DECISION_COMMITTED`	Decision activated
`error` / `auth_error`	Failure

UI file: `apps/web/src/app/(dashboard)/chat/ChatClient.tsx`

Coordinator agent (`agent.ts`)

- **System prompt:** Institutional reasoning, org context, active decision
- **Tools available:**
  - `log_strategic_decision` — create DRAFT decision + tasks
  - `transition_execution` — move task state
  - `delegate_to_marketing` / `delegate_to_research` — sub-agents (web search only)
  - Memory tools: `remember_fact`, `recall_memory`
  - `ping`, file ops, web search
- **Streaming:** Sends `text_chunk` events as Claude responds
- **Cost saver:** Default on — tighter history/iterations (`LLM_COST_SAVER=1`)

LLM model routing

Files: `apps/api/src/llm.ts`, `anthropic-model-policy.ts`, `config.ts`

Tier	Default model	Used for
Premium	`claude-sonnet-4-6`	Main coordinator
Fast	`claude-haiku-4-5-20251001`	Intelligence, sub-agents
OpenAI fallback	`gpt-4o-mini`	If `ACTIVE_PROVIDER=openai`

Env overrides: `ANTHROPIC_MODEL_PREMIUM`, `ANTHROPIC_MODEL_FAST`, `LLM_COORDINATOR_HAIKU=1` (cheaper coordinator)

Check live config: `GET /api/health` → `anthropicPremiumModel`, `anthropicFastModel`

Known design tension

Coordinator tools can **mutate the database directly** (convenient for demos). Target architecture: **propose** → **validate** → **commit** via API only. Documented in `docs/architecture-enforcement.md`.

---

12. Auditor — Deterministic Enforcement

File: `apps/api/src/auditor.ts`

The auditor is **not AI**. It runs fixed rules against database state.

What it checks

Rule	Meaning
Shadow work	Signals in GRACE period — unlinked work outside active decision
Capacity	Owner/task weight vs configured limits
NO-list	Blocked work patterns
Structural collapse	Dependency/orphan issues
Grace expiry	Auto-kill signals past grace window

When it runs

Trigger	When
After signal ingest	`runAfterSignalIngested()` — shadow work pass
After ledger mutation	`runAfterLedgerMutation()` — full evaluate on transitions, links, commits
Cron every 5 min	Safety net in `bootstrap.ts`

Conflict gate (one ACTIVE decision)

When activating a decision (`status → ACTIVE`):

- `findOtherActiveDecision(user_id)` checks for existing ACTIVE
- If found → throws `CONFLICT` , logs `DECISION_CONFLICT_BLOCKED`
- Applies to: `POST /api/decisions/commit` , WebSocket `COMMIT_DECISION`

DRAFT creation is **not** conflict-checked — only activation.

---

13. Intelligence Layer

Orchestrator: `apps/api/src/intelligence/index.ts`

When a signal arrives, this pipeline runs (async):

```
1. contextEngine    → fetch active decision + execution context
2. signalDetector   → LLM + heuristics: intent, risk, urgency
3. enrich           → persist classification on signal row
4. embeddings       → vector BLOB in entity_embeddings
5. autoLinker       → link to task if confidence ≥ threshold
```

Search

Hybrid search (`hybridSearch.ts`): combines FTS5 keyword search + embedding similarity.

API: `GET /api/intelligence/search?q=...`

Operational queries

Endpoint	Question answered
`/api/operational/drift`	Is execution drifting from the vow?
`/api/operational/overloaded`	Who is over capacity?
`/api/operational/conflicts`	What work conflicts?
`/api/reports/weekly`	Weekly drift markdown report

Cost gates

Skips LLM enrichment when: signal too small/noisy, no active decision, already enriched.

---

14. Integrations

Registry: `apps/api/src/auth/oauthProviders.ts`

Provider	Connect	Ingest
Slack	OAuth	Events API webhook → signals
GitHub	OAuth + repo select	Webhooks → signals
Google	OAuth	Calendar + Gmail workers
Microsoft	OAuth	Teams + Outlook subscriptions
Notion	OAuth	Poll worker
Jira	OAuth	Webhooks
Linear	OAuth	Webhooks

Normalized ingest path

All sources → `integrations/ingestRouter.ts` → `webhookTransforms.ts` → `external_signals` row → auditor → intelligence pipeline.

Background workers (started at boot)

- OAuth token refresh (`auth/oauthRefreshWorker.ts`)
- Microsoft/Jira subscription renewal
- Notion poll
- Google calendar/gmail sync
- Notification digest scheduler

Integration health

`GET /api/integrations/health` — connection status for Settings UI.

Roadmap gaps: `docs/integration-backlog.md`

---

15. Notifications

Dispatcher: `apps/api/src/notifications/dispatcher.ts`

When something important happens (decision committed, signal grace expiring, etc.):

1. Create row in `notifications` table
2. Check user preferences from `settings` (category toggles, channels)
3. Deliver via enabled channels:
   - **In-app** — always stored
   - **Email** — immediate SMTP (`auth/smtp.ts`)
   - **Slack DM** — via bot token from `oauth_connections`
   - **Digest** — batched by `notifications/digest.ts` scheduler

Settings UI maps toggles to `PREF.*` keys via `POST /api/settings/bulk`.

---

16. Deployment & Infrastructure

Production target: Azure Container Apps

- **URL:** `app.hustronix.online`
- **Config:** `infra/azure/` (Bicep, Key Vault, ACR)
- **Deploy:** `./infra/azure/deploy.sh` or CI on `v*` tags (`.github/workflows/deploy-azure.yml`)
- **Marketing:** `hustronix.online` on Netlify (`netlify-landing/`)

Docker

```
docker build -f infra/docker/Dockerfile -t hustronix .
docker run --env-file .env -p 3000:3000 -v hustronix-data:/app/data hustronix
```

Or: `docker compose -f infra/docker/docker-compose.yml up --build`

Fly.io

```
fly deploy -c infra/fly/fly.toml
```

Persistent volume for SQLite at `/app/data`.

CI pipelines

Workflow	Purpose
`.github/workflows/build.yml`	Build all workspaces
`.github/workflows/qa-ui.yml`	UI QA pass
`.github/workflows/deploy-azure.yml`	Production deploy on tags

Production checklist (from README)

1. `NODE_ENV=production`
2. Change `MISSION_CONTROL_API_KEY` from example
3. Set `ALLOWED_ORIGINS` and `APP_ORIGIN` to exact HTTPS URLs
4. `REQUIRE_HTTPS=true` behind TLS
5. Set `OAUTH_TOKEN_ENCRYPTION_KEY` and `TOTP_ENCRYPTION_KEY`
6. Configure LLM keys (`ANTHROPIC_API_KEY`)
7. Wire integration OAuth credentials per provider
8. Mount persistent storage for `SQLITE_PATH`
9. Run `npm run security:launch`
10. Run `npm run build` and `npm run qa:ui`

---

17. Environment Variables Reference

Root `.env` (API + server)

Variable	Purpose	Example
`PORT`	HTTP port	`3005` (dev) / `3000` (prod unified)

Variable	Purpose	Example
`ALLOWED_ORIGINS`	CORS allowed browser origins	`http://localhost:3000`
`APP_ORIGIN`	Public UI URL for OAuth redirects	`https://app.hustronix.online`
`SQLITE_PATH`	Database file path	`./data/hustronix.db`
`MISSION_CONTROL_API_KEY`	Script/automation key	(change in prod)
`ANTHROPIC_API_KEY`	Claude API key	`sk-ant-…`
`ANTHROPIC_MODEL_PREMIUM`	Coordinator model	`claude-sonnet-4-6`
`ANTHROPIC_MODEL_FAST`	Fast tier model	`claude-haiku-4-5-20251001`
`LLM_COORDINATOR_HAIKU=1`	Force cheap coordinator	optional
`LLM_COST_SAVER=0`	Disable cost caps	default on
`OPENAI_API_KEY`	OpenAI fallback	if using OpenAI
`OAUTH_TOKEN_ENCRYPTION_KEY`	Integration token encryption	64 hex chars
`TOTP_ENCRYPTION_KEY`	2FA secret encryption	64 hex chars
`REQUIRE_HTTPS`	Force HTTPS	`true` in prod
Per-provider OAuth	`SLACK_*`, `GITHUB_OAUTH_*`, etc.	see `.env.example`
SMTP	`SMTP_*` for email OTP/notifications	optional

`apps/web/.env.local` (Next.js)

Variable	Purpose
`NEXT_PUBLIC_API_URL`	Absolute API URL, or empty for same-origin `/api`
`API_PROXY_TARGET`	Where Next proxies `/api/*` (default `http://127.0.0.1:3005`)
`NEXT_PUBLIC_WS_URL`	WebSocket URL (default derives from API)
`NEXT_PUBLIC_API_KEY`	Dev automation only (with `hustronix_userId` in localStorage)

Full list: `.env.example` and `apps/web/.env.example`

---

18. Product Maturity — What's Real vs Partial

Fully wired (live API + UI)

- Mission Control, Execute, Signals, Chat, Reports, Teams
- Profile (except password form UI — backend exists)
- Settings (General + Notifications; some tabs UI-only)
- Auth: password, OTP, OAuth login, sessions, API keys, TOTP backend
- Integrations: OAuth connect + webhook ingest for major providers
- Auditor + conflict gate + intelligence pipeline
- Notifications: in-app + email/Slack/digest

Partial / mock / backlog

Area	Status
Profile password change UI	Backend `POST /api/auth/password` exists; form UI incomplete
Settings Organization/Rules/Capacity tabs	Some UI-only, not all persisted
`connections_manager` capability cards	Partially demo/mock

Area	Status
PostgreSQL runtime	Schema ready; still SQLite in production
Coordinator direct DB writes	Works; target is propose→validate→commit
Telegram bridges	Removed from repo
Billing	Stub endpoints

Scope doc: `docs/product-scope.md` — direction is **demo-full** with enforcement kernel as truth.

---

19. Glossary

Term	Definition
Decision	A strategic commitment (the "vow") with outcomes and constraints
Execution task	A unit of work linked to a decision
Signal	An external event (Slack msg, GitHub PR) ingested into the ledger
GRACE	Time window where unlinked signal is visible before auto-kill
LINKED	Signal explicitly tied to an execution task
KILLED	Signal dismissed (by user or auditor)
ACTIVE decision	The one committed decision currently in force (max 1 per org)
DRAFT decision	Proposed decision not yet committed
Coordinator	The main LLM agent in chat
Auditor	Rule engine — not AI
Tenant	One customer org, scoped by `user_id`
Unified server	Single process serving API + UI + WebSocket

20. Key Documentation Index

Document	Topic
`README.md`	Quick start, commands, deploy
`docs/architecture-enforcement.md`	Ledger vs LLM, auditor triggers
`docs/security-architecture.md`	Threat model, target hardening
`docs/product-scope.md`	Demo-full vs minimal scope
`docs/agents-and-slack-signals.md`	Agent + Slack flow
`docs/integration-backlog.md`	Integration roadmap
`.cursor/rules/hustronix-stack.mdc`	Developer stack contract

Appendix A — Decision State Machine

```
DRAFT ──commit──► ACTIVE ──► COMPLETED / ARCHIVED / SUPERSEDED
                     │
                     └── blocked if another ACTIVE exists (conflict gate)
```

Appendix B — Signal State Machine

```
UNVERIFIED ──► GRACE ──link──► LINKED
                 │
                 └──kill──► KILLED (user or auditor after grace expiry)
```

Appendix C — Common Dev Commands

```
npm install                          # Install all workspaces
cp .env.example .env                 # API env
cp apps/web/.env.example apps/web/.env.local
```

```
npm run dev                              # API :3005 + web :3000
npm run build                            # Build shared + api + web
npm run start:unified                    # Production unified server
npm run security:launch                  # Security checklist
npm run qa:ui                            # UI QA pass
curl http://localhost:3000/api/health
```

This guide reflects the codebase as of June 2026. For the latest deploy steps, see `README.md`. For enforcement design rationale, see `docs/architecture-enforcement.md`.